VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Project Report

Topic: Searching for the Knapsack

Subject: Introduction to Artificial Intelligence

Students:
Hoang Anh Tra (21127453)
Nguyen Thanh Binh (21127232)
Truong Vinh Phuc (21127673)

Instructors:
Nguyen Tien Huy
Nguyen Tran Duy Minh
Bui Duy Dang

July 21, 2023

Contents

1 Work Assignment **2**
1.1 Member list . . . 2
1.2 Assignment . . . 2
1.3 Work completeness . . . 2

2 Algorithms' description **2**
2.1 Brute-force search . . . 2
2.1.1 Idea . . . 2
2.1.2 Pseudocode . . . 3
2.1.3 Visualization . . . 3
2.2 Branch and Bound . . . 4
2.2.1 Idea . . . 4
2.2.2 Pseudocode . . . 5
2.2.3 Visualization . . . 6
2.3 Local beam Search . . . 7
2.3.1 Idea . . . 7
2.3.2 Pseudocode . . . 8
2.3.3 Visualization . . . 9
2.4 Genetic algorithm . . . 9
2.4.1 Idea . . . 9
2.4.2 Pseudocode . . . 10
2.4.3 Visualization . . . 12

3 Evaluation **12**
3.1 Evaluation methods . . . 12
3.2 Data sets . . . 12
3.3 Result and comment . . . 13
3.3.1 Brute-force . . . 13
3.3.2 Branch and Bound . . . 14
3.3.3 Local Beam . . . 15
3.3.4 Genetic Algorithm . . . 16

4 Conclusion **16**
4.1 What we learn . . . 16
4.2 How we felt . . . 17

5 Demo Video **17**

References **18**

1 Work Assignment

1.1 Member list

Student ID	Name	Role
21127232	Nguyen Thanh Binh	Member
21127453	Hoang Anh Tra	Leader
21127673	Truong Vinh Phuc	Member

.

1.2 Assignment

Id	Task	Name	Status
1	Research and implement "Brute-force searching" algorithm	Nguyen Thanh Binh	Complete
2	Research and implement "Brand and bound" algorithm	Truong Vinh Phuc	Complete
3	Research and implement "Local beam search" algorithm	Hoang Anh Tra	Complete
4	Research and implement Genetic algorithm	Nguyen Thanh Binh	Complete
5	Generate test cases	All of members	Complete
6	Summarize and complete report	Hoang Anh Tra	Complete
7	Create and edit video	Truong Vinh Phuc	Complete

.

1.3 Work completeness

Id	Work	Completeness
1	Implementation for all algorithms	100%
2	Generating and executing all algorithms on small and big test cases	100%
3	Making videos running all data sets	100%
4	Improvement few parts for some algorithms	100%

.

2 Algorithms' description

2.1 Brute-force search

2.1.1 Idea

- **The brute force algorithm** is a straightforward but inefficient method to find the optimal solution. The idea of it is to try every possible combination to satisfy the constraints and return the best solution at the end. For the knapsack problem, it tries all possible combinations

of items to find the one that maximizes the value of the knapsack without exceeding its capacity and having enough classes.

- In this project, we have implemented this algorithm in a recursive way. We will go through each item and run both of two ways, the first is putting this item into the knapsack and the second is not. In each way, we will save the current combination if it is better than all of the previous ones. When we go through all of the items, we will have the best solution.

2.1.2 Pseudocode

`Algorithm 1: Brute-force Algorithm`

function BRUTE-FORCE(problem, index, get-item) return void
  cumulations:
    total-value, total value of the current combination
    total-weight, total weight of the current combination
    max-total-value, max of total value
    knapsack, list of item in knapsack
    max-knapsack, result list of item in knapsack
  inputs:
    problem, knapsack problem
    index, index of an item in the knapsack
    get-item, decision of putting the item into the knapsack
  if get-item **then**
    total-value ← total-value + problem.values.AT(index)
    total-weight ← total-weight + problem.weights.AT(index)
    knapsack.AT(index) ← 1
    if total-weight < problem.W and total-value > max-total-value and number of class of the current combination is enough **then**
      max-total-value ← total-value
      max-knapsack ← knapsack
      BRUTE-FORCE(problem, index + 1, True)
      BRUTE-FORCE(problem, index + 1, False)
      total-value ← total-value - problem.values.AT(index)
      total-weight ← total-weight - problem.weights.AT(index)
      knapsack.AT(index) ←0
  else then
    BRUTE-FORCE(problem, index + 1, True)
    BRUTE-FORCE(problem, index + 1, False)

2.1.3 Visualization

- We use a simple data set to illustrate this algorithm. This data set has the maximum weight (W) is 16, number of classes (m) is 2, number of items (n) is 3, list of weights (w) is [12, 7, 3], list of values (v) is [23, 12, 10] and the list of classes (c) is [1, 1, 2].

- Here is our visualization of Brute Force algorithm:

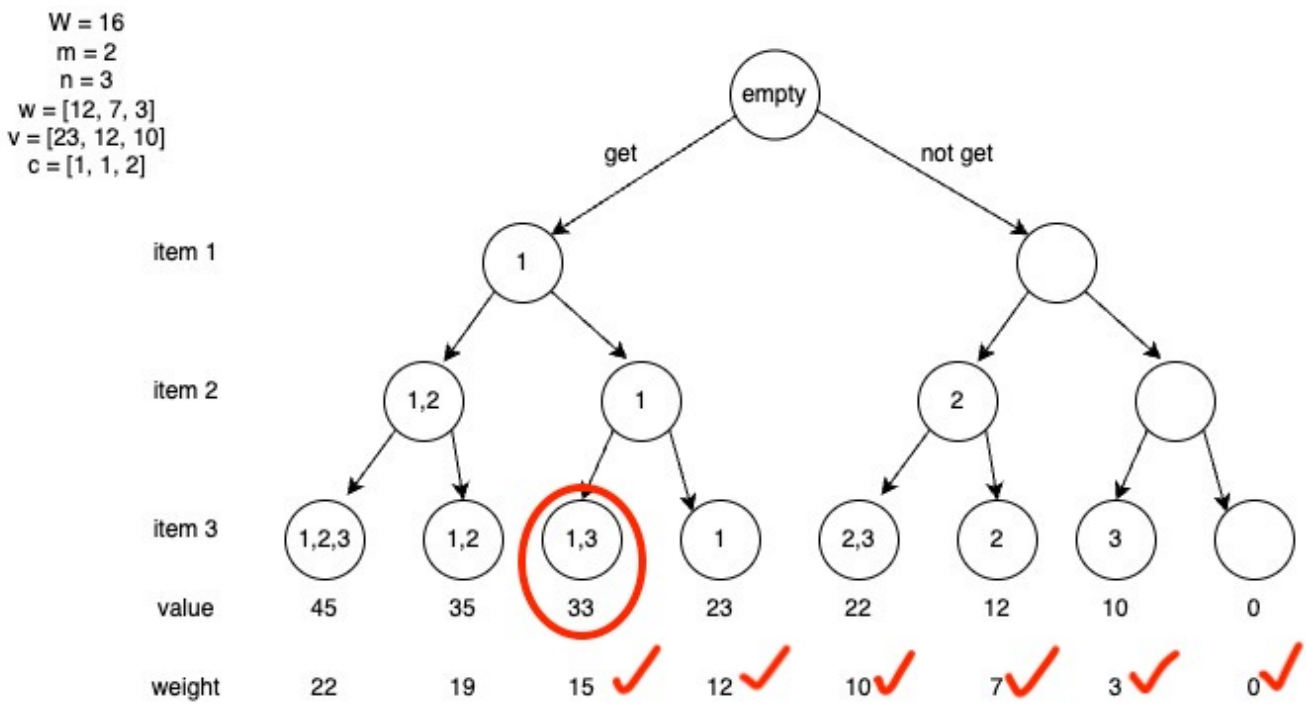

Figure 1: Visualization of the Brute Force algorithm

- We run this algorithm like the DFS way in tree traversing. It includes some following steps:
  - Step 1: From level 0 to level 3 of this tree, we decide the first way which is putting all items into the knapsack.
  - Step 2: We backtrack to the previous level and decide another way which is not putting this item into the knapsack.
  - Step 3: Loop step 1 and step 2 over and over again until we try all branches of this tree. Each time we are at level 3, we have to calculate the value and check if it is valid or not and save it if it is the best value so far.
  - Step 4: After running all of the possibilities and evaluating all of them, we have the result value is 33 with the weight 15 ¡ 16 and the number of classes is equal to 2. This result is marked by a red circle.

2.2 Branch and Bound

2.2.1 Idea

- [4] We use the backtracking algorithm and set the upper bound function g to eliminate the cases that cannot be the most optimal solution.
- To solve this problem, we must determine the upper bound function and that is using a fractional knapsack algorithm.
- If the current value is greater or equal than the bound, we prune the branch then try another item. Else we try the item and count it.

- Check if all the classes have been selected, we return the current value and weight else return 0.

Notice:

- After doing the function **knapsack_recursive_BNB**, we must check again if all classes have been selected. If no, return no solution.
- As I use the recursion method, it may not be the best function to work with large test cases with a big number of items or the number of items is medium and the capacity is too large for the weight of those items. For example: Capacity is 500, but the weight is around [1,5].
- I don't use the best heuristic function to calculate the upper bound so it will take a long time to run such big data with n > 100

2.2.2 Pseudocode

```
Algorithm 2:  Branch and Bound Algorithm [1]
Total_weight, Items_weight, Items_value, Items_class, max_value = 0, selected_items
n = len(Items_weight)
Function upper_bound(index, current_weight, current_value):
    Remaining_weight ← Total_weight - current_weight
    Total_value ← current_value
    sort_indices
    for i in sort_indices:
        if Items_weight[i] <= remaining_weight then
            total_value ← total_value + Items_value[i]
            remaining_weight ← remaining_weight - Items_weight[i]
        else then
            total_value ← total_value + (remaining_weight / Items_weight[i]) * Items_value[i]
            end for
    return total_value
Function knapsack_recursive_BNB(current_value, current_weight, index, selected_items,
max_value, max_items):
    if index == n then
        if current_value > max_value then
            return current_value, selected_items
        else then
            return max_value, max_items
    bound ← upper_bound(index,current_weight,current_value)
    if bound <= max_value then
        return max_value, max_items
    try to select the branch
    try not to select the branch
    if all class not selected then
        choose the item that has the class
    else then
        return max_value, max_items
```

2.2.3 Visualization

The test case:

- W = 10
- m = 2
- w = 4, 7, 5
- v = 40, 42, 25
- c = 1, 1, 2

```
Not selecting items 0
Not selecting items 1
Not selecting items 2
Try selecting items 2
Try selecting items 1
Not selecting items 2
Try selecting items 2
Not selecting items 2
Try selecting items 2
Try selecting items 0
Not selecting items 1
Not selecting items 2
Try selecting items 2
Try selecting items 1
Not selecting items 1
Try selecting items 1
Maximum value: 65
Selected items:
Item 0 chosen 1 time(s)
Item 2 chosen 1 time(s)
```

Figure 2: Visualization of the Branch and bound algorithm

Algorithms can be performed like this image (notice: this visualization does not perform all moves of the algorithm)

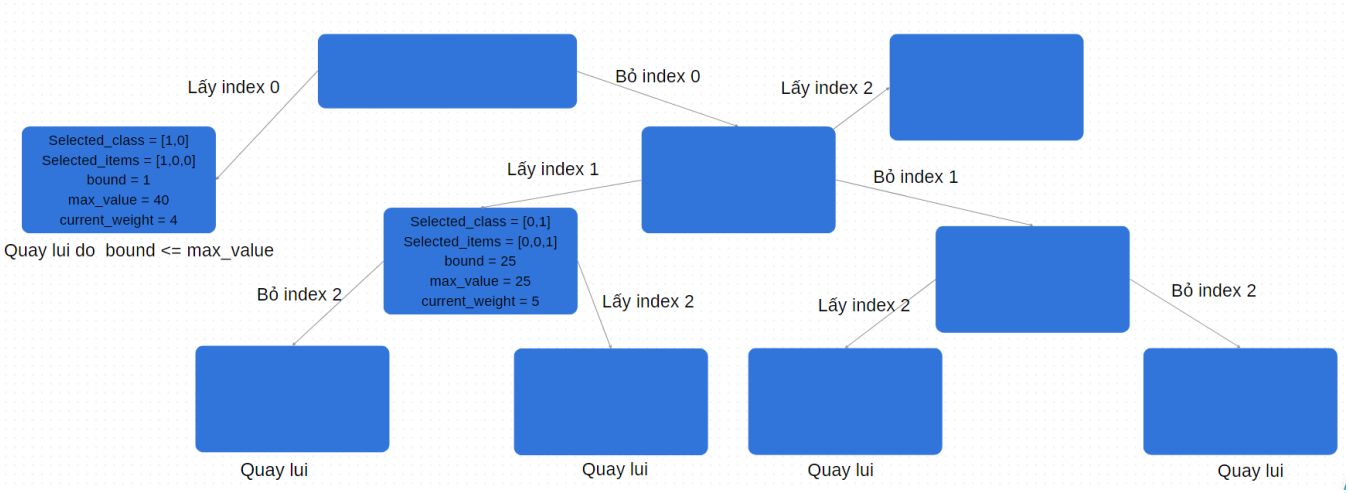

Figure 3: Visualization of the Branch and bound algorithm

- First, the algorithm exclude index 0, 1 and 2 then return max_value, max_items as index == n and backtrack
- After backtracking, it will include the index 2 then backtrack again.
- Notice how the algorithm prunes the branch index 0 at first. As the upper_bound() function is calculated and compared to the current max_value, the result is bound <= max_value.
- Keep following the flow like that, the maximum value is 65 and the selected items are item 0 and item 2

2.3 Local beam Search

2.3.1 Idea

- Local beam search [7] is a heuristic search algorithm that is used to solve optimization problems such as the knapsack problem. The idea behind local beam search is to use multiple starting points to explore the search space more effectively than a single starting point.
- The local beam search algorithm keeps track of k states simultaneously. Initially, k randomly generated states are generated. Since we can't tell if the given state is the goal or not in this Knapsack problem, the algorithm will only loop a max iterator number of times and then return the best state it has found. At each loop, all the successors of the k states are generated (neighbor) and put into a priority queue (neighborhood). Then the algorithm selects the k best successors from the queue by popping it k times.
- In Particular, the algorithm will follow the number of classes of each solution. If the number of classes of the final solution is not equal to m class, the algorithms will return the failure value (full of 0).

2.3.2 Pseudocode

`Algorithm 3: Local Beam Algorithm` [3] [8]

```
function localbeam(k, max_iter)
    states ← generate-k-states(k)
    initial-solution ← state-with-max-value(states)
    beams ← list-with-k-initial-solution
    best-solution ← initial-solution
    best-value ← evaluate(best-solution)
    neighborhood ← empty-priority-queue
    for i ∈ [0, max_iter - 1] do
        neighborhood ← neighborhood.enqueue(generate-neighborhood(beams))
        beam ← select-k-max-value(neighborhood)
        new-best-solution ← max-solution(beams)
        new-best-value ← evaluate(new-best-solution)
        if new-best-value ¿ best-value then
            best-solution ← new-best-solution
            best-value ← new-best-value
        end if
    end for
    if number-class-of-solution ¡ m-class then
        return no-solution
    end if
    return best-solution
end function
```

2.3.3 Visualization

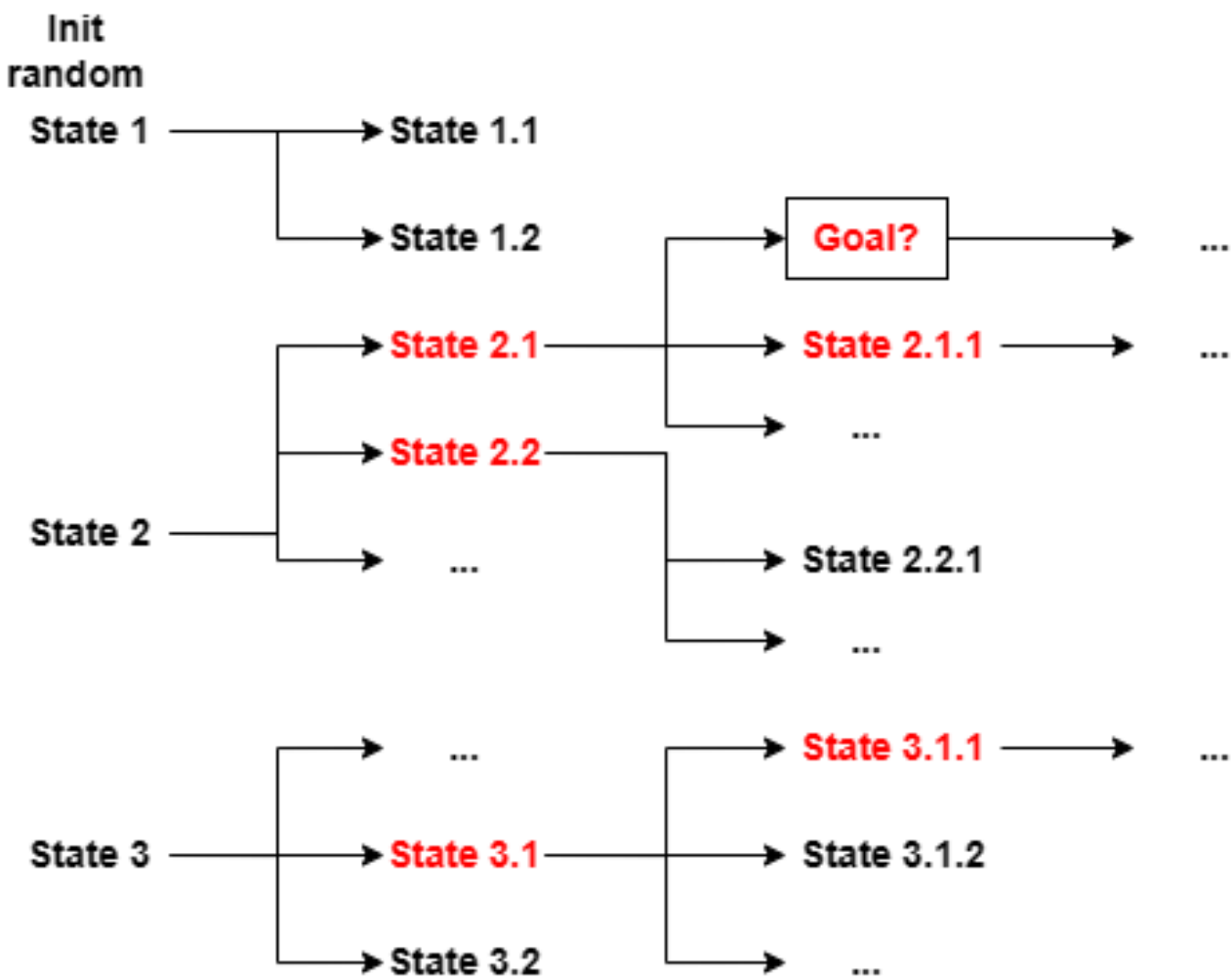

Figure 4: Visualization of the Local beam search algorithm

- First, we will initialize the initial k states (In this example, k = 3). From those k states, we create sub-states (neighbors) and put them in the priority queue. Pick the k states with the largest value and so on until the target is found or the number of iterations is exhausted.
- In fact, we never know whether the current state is optimal or not, and we also do not know the maximum number of iterations to get the optimal state.

2.4 Genetic algorithm

2.4.1 Idea

- The genetic algorithm (GA) [5] is a metaheuristic inspired by the process of natural selection. It works by iteratively generating a population of solutions to the problem, and then selecting the best solutions to create a new population. This process is repeated until a stopping criterion is met, such as a maximum number of iterations or a certain level of fitness.

- In this project, the GA can be used to solve the knapsack problem by representing each solution as a chromosome. A chromosome is a binary string that encodes the decision of whether or not to include each item in the knapsack. The fitness of a chromosome is a measure of how good it is, and it is typically calculated as the total profit of the items included in the knapsack.
- Crossover is a genetic operator that combines two chromosomes to create a new chromosome. Mutation is a genetic operator that randomly changes a chromosome. The use of genetic operators helps to introduce new diversity into the population, which can help the GA to find better solutions.

2.4.2 Pseudocode

`Algorithm 4: Genetic Algorithm` [9] [2]

```
function GENETIC-ALGORITHM(problem) return void
    cumulations:
        population, the population
        generations, number of generations
        mutation-rate, mutation rate of an individual
        best-value, best value of population
        best-individual, the individual which has best value
        result-value, result value of the result individual
        result-individual, the result individual which will be written to the output file.
        fitness-scores, list of fitness score of all individuals
        fitness-ratio, list of fitness ratio
    inputs:
        problem, knapsack problem
    population = INIT-POPULATION(number of items in population)
    for i in range of generations
        new-population ← empty state
        SORT(population, key = FITNESS-FUNCTION)
        new-population ← new-population + k individuals of population
        fitness-scores ← FITNESS-FUNCTION for each of individual in population
        fitness-ratio ← fitness-scores / sum of fitness-scores
        for i in range of generations / 2 - k
            parents = SELECTION(population, fitness-ratio)
            child1, child2 ← CROSS-OVER(parent1, parent2)
        child1 ← MUTATION(child1, mutation-rate)
        child2 ← MUTATION(child2, mutation-rate)
        new-population.EXTEND(child1 and child2)
        population ← new-population
        best-value, best-individual ← GET-BEST-INDIVIDUAL(population)
        if best-value > result-value then
            result-value ← best-value
            result-individual ← best-individual
```

- The **INIT-POPULATION** initializes some of the individuals which have to be valid for problem requirements, if there aren't any valid individuals, the function will return and won't do anything else.
- The **FITNESS-FUNCTION** calculates the total profit of the items included in the knapsack. If the weight of the knapsack is greater than the capacity or the number of classes is not enough, then the fitness is set to 0. Otherwise, the fitness is equal to the total profit.
- The **SELECTION** selects two parents based on the ROULETTE WHEEL SELECT that prefers choosing the individual with high fitness-ratio.
- The **CROSS-OVER** takes two chromosomes as input, and it produces two new chromosomes as output. The function first randomly selects a point in the chromosomes, and then it swaps the segments of the chromosomes after the selected point. The function then returns the two new chromosomes.
- The **MUTATION** takes a chromosome as input, and it produces a new chromosome as output. The function first randomly determines if a bit in the chromosome will be flipped. If a random probability is smaller than the mutation rate, the function flips the bit. The function then returns the mutated chromosome.
- The mutation rate can be changed based on the population improvement, if the population is not improved for a while, the mutation rate will be increased and decreased later when the population is better.

2.4.3 Visualization

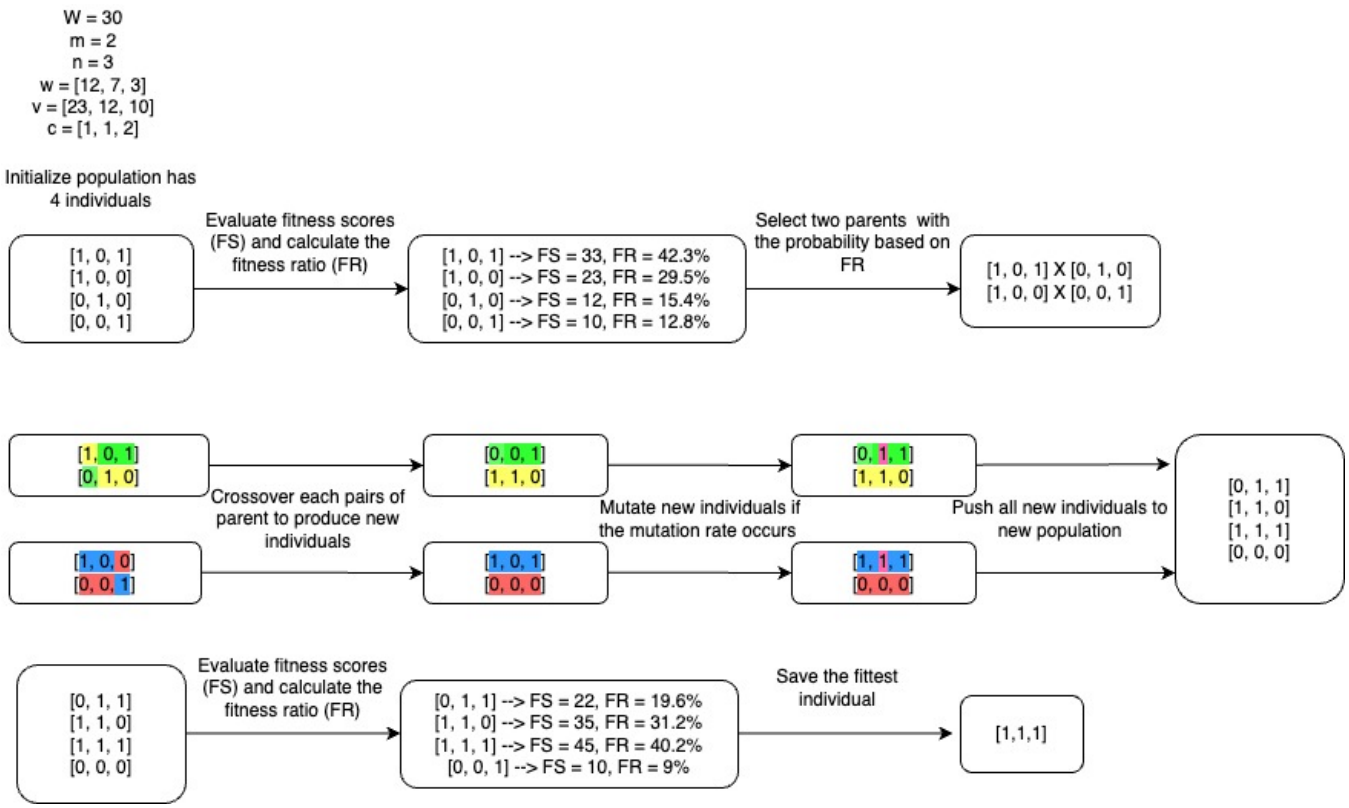

Figure 5: Visualization of the genetic algorithm

3 Evaluation

3.1 Evaluation methods

- With the 4 algorithms implemented above, we will test 12 test cases in turn with a time limit for each small test case of 60 seconds and for each big test case of 120 seconds by using `time` and `signal` modules.
- After the above time limit, if the algorithm does not give results, it will write to the output file as "RUN OUT OF LIMITED TIME".

3.2 Data sets

There are in total 12 data sets. The first 6 data sets, from Input_1 to Input_6, are small data sets, while the remaining are large ones.
Below are the data sets with their notes.

INPUT	N (number of items)	M (number of classes)	W (Capacity)	Max value (MaxTotalV)	Note
1	21	2	668	901	Normal
2	10	3	487	589	Pick full of items
3	10	3	165	No solution	No solution because the number of classes is not enough
4	24	5	6404180	13549094	Big number
5	38	2	137	642	Normal
6	16	2	3493	No solution	No solution because the number of classes is not enough
7	554	7	410	3719	Many items but the max weight is quite small
8	93	8	132	835	Normal
9	61	5	9821	3240	Pick all of items
10	193	5	7314	9049	Normal
11	1000	7	881	5137	Normal with max number of items
12	397	5	60	677	Just can pick some first elements because the max weight is small

Figure 6: Data sets

3.3 Result and comment

3.3.1 Brute-force

- **Result**:

Algorithm 1: Brute-force												
Size	Small						Big					
Input	1	2	3	4	5	6	7	8	9	10	11	12
MaxTotal V	901	589	No Solution	13549094	N/A	No Solution	N/A	N/A	N/A	N/A	N/A	N/A
Execution time (seconds)	1.904	0.001	0.001	15.476	60.004	0.064	120.002	120.002	120.009	120.017	120.006	120.003

Figure 7: Testing result of the brute-force algorithm

- **General comment**:
  - The Brute Force method examines every scenario for selecting and excluding items for the knapsack. It will therefore always arrive at the best answer.
  - The algorithm is simple to develop and simple to use.
  - The approach is vulnerable to an exponential explosion, though, as the number of potential choices that the program must weigh doubles for each item that is added to the problem. Thus, even problems of a somewhat small size cannot be solved using the Brute Force approach.

- Although the Brute Force algorithm always selects the best combination of items, the time trade-off is so great that even reasonably small-size Knapsack issues become insoluble. As a result, it is not a sophisticated solution to this issue.

- **Advantages**:
  - The brute-force algorithm is straightforward to implement, making it easy to understand and debug.
  - Since the brute-force algorithm exhaustively examines all possible solutions, it guarantees to find the optimal solution if one exists.

- **Disadvantages**:
  - The brute-force algorithm has a time complexity that grows exponentially with the input size. Therefore, it becomes impractical or even infeasible when dealing with large problem instances.
  - The algorithm explores all possible solutions, including those that are obviously suboptimal. This leads to a lot of redundant computations and wasted time.

- **Time complexity**: $O(2^n)$ because the algorithm must try every possible combination of items to find the optimal solution.

- **Space complexity**: $O(n)$

3.3.2 Branch and Bound

- **Result**:

Algorithm 2: Branch and Bound												
Size	Small						Big					
Input	1	2	3	4	5	6	7	8	9	10	11	12
MaxTotal V	901	589	No Solution	13549094	642	No Solution	N/A	835	3240	N/A	N/A	N/A
Execution time (seconds)	0.423	0.001	0.002	3.189	0.305	0.002	120	38.131	0.119	120	120	119

Figure 8: Testing result of the branch and bound algorithm

- **General comment**:
  - The algorithm seems to perform well in the first six instances, where the input sizes are relatively small. However, the algorithm's execution time significantly increases for inputs, where the maximum weight is much larger. This suggests that the algorithm's performance deteriorates as the input size increases, which is expected given its exponential time and space complexity.
  - Compared to brute force: Both have the same time complexity although branch and bound runs faster than brute force. Therefore, the branch and bound algorithm should be preferred over the brute force algorithm when the number of items is large, as it can significantly reduce the search space and improve the running time.

- **Advantages**:
  - Save time by pruning the ineffective branch.
  - If the problem is small, the time is not considerable.
- **Disadvantages**:
  - In the worst case or big data: the branch and bound runs very slow
  - As it backtracks a lot, the stack in the system can overflow.
- **Time complexity**: $O(n.\log n)$
- **Space complexity**: $O(n)$

3.3.3 Local Beam

- **Result**:

Algorithm 3: Local Beam												
Size	Small						Big					
Input	1	2	3	4	5	6	7	8	9	10	11	12
MaxTotal V	758	589	No Solution	12596395	No Solution	No Solution	N/A	No Solution	3240	8747	N/A	No Solution
Execution time (seconds)	0.385	0.126	0.106	0.496	0.853	0.254	120	5.665	3.552	30.904	119.995	97.929

Figure 9: Testing result of the local beam algorithm

- **General comment**:
  - The advantages of the local beam search algorithm for knapsack problems are that it is a simple and efficient algorithm that can be used to solve optimization problems such as the knapsack problem. It is also easy to implement and can be used to find good solutions quickly.
  - However, The algorithm can suffer from a lack of diversity among the k states which can become clustered in a small region of the state space, making the search little more than a k-times-slower version of hill climbing. A variant called stochastic beam search helps alleviate this problem. [7]
  - In addition, randomly generated initial states make it much harder to find an optimal solution. To be able to find the most optimal solution, it is necessary to come up with initial solutions that must be closest to the goal or must be the most comprehensive. Therefore, we can use permutations to give the best initial state, but this also leads to a lot of time and space costs.
- **Time complexity**: $O(k.n)$ with k is the number of intial states.
- **Space complexity**: $O(i.k.n)$ with i is the max iterator.

3.3.4 Genetic Algorithm

- **Result**:

Algorithm 4: Genetic Algorithm												
Size	Small						Big					
Input	1	2	3	4	5	6	7	8	9	10	11	12
MaxTotal V	901	589	No Solution	13549094	636	No Solution	3502	792	3240	9208	N/A	677
Execution time (seconds)	4.157	2.943	0.001	12.209	5.588	0.001	87.326	13.648	9.057	69.371	120	53.03

Figure 10: Testing result of the genetic algorithm

- **Advantages**:
  - The genetic algorithm has flexibility because it can be used to solve a wide variety of knapsack problems, including problems with a large number of items and a small capacity. [6]
  - The genetic algorithm can be scaled to solve larger and more complex knapsack problems.
- **Disadvantages**:
  - The genetic algorithm can be computationally expensive, especially for large knapsack problems.
  - The performance of the genetic algorithm can be sensitive to the choice of parameters, such as the mutation rate and the crossover rate.
  - The genetic algorithm may not converge to the optimal solution, especially for large knapsack problems.
- **Time complexity**: Exponential in the number of items.
- **Space complexity**: Exponential in the number of items.

4 Conclusion

4.1 What we learn

- Finally, we examined and compared four different algorithms for solving the knapsack problem. Brute Force, Branch and Bound, Local Beam Search, Genetic Algorithm. Each algorithm has its own strengths and weaknesses, and the best choice depends on the particular problem instance and the desired compromise between computation time and solution quality.
- Brute force algorithms guarantee the best solution, but quickly become infeasible for large problems. Branch-and-bound can efficiently handle larger problem sizes by pruning the search tree using the bound function. However, the quality of your solution depends on your choice of bound function. A local ray search starts with some initial solutions and iteratively improves by examining neighboring solutions. A good solution is often found quickly, but it is possible

to get stuck in a local maximum. Genetic algorithms apply the concept of evolution to iteratively generate and select a population of candidate solutions in search of the optimal solution. This allows efficient exploration of large regions of the search space, but can require significant computational time.

- Overall, algorithm choice depends on problem size, time constraints, and desired quality of solution. No algorithm is universally good, but understanding the strengths and weaknesses of each algorithm can help you choose the most appropriate approach for your particular problem instance.

4.2 How we felt

- Working on this project was an interesting and exciting experience for our group. It enabled us to have a deeper comprehension of different algorithms and how they are used to address optimization issues. We were eager to investigate several solutions to the knapsack problem and evaluate their effectiveness.

- We had some difficulties along the road, such as debugging and troubleshooting, but overall we found the process to be pleasant and instructive. We are thankful for the opportunity to work on the project because it allowed us to gain useful expertise in algorithm analysis and optimization. We are excited to use the knowledge we learned from this project and believe it will be helpful in our future undertakings.

5 Demo Video

This is the Playlist of Testing Progress.

References

[1] Inside Code. 0-1 knapsack problem. *youtube.com*, 2022.

[2] Computerphile. The knapsack problem genetic algorithms. *youtube.com*, 2020.

[3] faithdanghuy. Knapsack-artificial-intelligent. *github.com*, 2023.

[4] Greeks For Geeks. Fractional knapsack problem. *geeksforgeeks.com*, 2023.

[5] Matthew Mayo. Genetic programming in python: The knapsack problem. *KDnuggets in Machine Learning*, 2023.

[6] Tutorials Point. Genetic algorithms - termination condition. *tutorialspoint.com*, 2023.

[7] Stuart Russell and Peter Norvig. Artificial intelligence: A modern approach, 4th us ed. *aima-115.pdf*, 2020.

[8] HCMUTE Team. Midterm report: Local beam search. *studocu.com*, 2023.

[9] Muhammad Saad Uddin. How to solve the knapsack problem using genetic algorithm in python. *plainenglish.io*, 2023.